

LECTURE - 14

(online)
~~Thursday~~ Tuesday
24/3/28.

□ CHIP : PROCESS SYNCHRONIZATION :

⇒ Shared memory : Normally every program has its own memory space. **BUT** sometimes processes need a shared memory space.

- Whenever memory is shared, OS does not have responsibility to manage them anymore; processes do it themselves and it's their responsibility not to write in the memory at "exact same time".

⇒ Process synchronization : In shared memory; it is necessary so any processes won't change memory at same time else there will be wrong results.

□ Producer-Consumer : Producer process produces information that is consumed by consumer process. e.g. : web server produces web pages for clients.

- There is shared memory used and for that there could be two types of Buffers:

1) Bounded Buffer : Fixed size. If buffer is full, producers must wait (no place to add produced process). If buffer is empty, consumer must wait (no process to consume).

2) UnBounded Buffer : No limit on size. Producer can produce as many process without waiting. Consumer need to wait if buffer is empty.

□ Race Condition : When counter value is wrong than expected bcs of producer & consumer using it in shared memory space.

How Counter is incremented/decrement in memory? Counter = 5

Step 1: Register = Counter's value Register = 5

Step 2: Register ++ Register = 6

Step 3: Counter = Register Counter = 6.

Now since in shared memory work is done concurrently; therefore if both producer & consumer increment & decrement simultaneously like below counter will be = 4/6 instead of 5: Counter = 5

T₁: Producer → R₁ = Counter R₁ = 5
 T₂: Producer → R₁ ++ R₁ = 6
 T₃: Consumer → R₂ = Counter R₂ = 5 and consumer intercepted
 T₄: Consumer → R₂ -- R₂ = 4 as counter not incremented
 T₅: Producer → Counter = R₁ Counter = 6
 T₆: Consumer → Counter = R₂ Counter = 4

* Race is b/w T₅ & T₆; which ever is executed first the other one will update counter as per its previous processing.

* Therefore to solve this problem the sequence of processes should be: T₁, T₂, T₅, T₃, T₄, T₆ giving counter = 5.

Critical Sec. for Producer → Critical Sec. for consumer.

⇒ Critical Section (CS): Part of code where process uses shared memory (like: counter variable) & changes it.

* There should be only one process in critical section parts like: either process or consumer so we don't have problems like race condition.

⇒ Remainder Section: Part of code with no shared memory (safe).

3 requirements in code to overcome problems.

① Mutual Exclusion: Only one process should be in CS at a time so there is no "race condition" problem.

② Progress: If no process is in CS, a new process must be added in it if it wants to so there are no "deadlocks" (unless waiting).

③ Bounded Waiting: any specific process who wants to enter CS must be given a chance to enter after ~~specific~~ limited no. of turns so there is "no starvation".

For all three algorithms:

i = current process no. } relative
 j = other process no. }

Process	i	j
P1	0	1
P2	1	0

} code for P1 will have i=0, j=1
 } code for P2 will have i=1, j=0.

Algo 1. Take Turns: (Solution)

do { // turn = 0 first for P1 to enter first
 while (turn != i); // if turn ≠ 0; then P1 will wait as P2 is alr. in CS.
 // critical section

turn = j // after using CS; process will force to give CS to other process even if
 // Remainder Section it does not need it.

} while (1) // infinite

Mutual exclusion: ✓ as only one process at a time per of "turn = i".

Progress: X If P1 leaves CS and gives P2 the hold of CS by doing "turn = j" and P2 doesn't need CS; so P1 will keep waiting if it needs CS again until P2 wants CS & actually uses it.

Bounded waiting: ✓ Alternate turns as "turn = j" (turn forcefully given).

Algo 2. Raise your hand.

do {
 flag[i] = true;
 while (flag[j] == true) // CS
 // RS
 flag[i] = false;
 // RS
 } while (1)
 Problem of forcefully giving control to other process is solved which Algo 1 had by using bool flag.
 → tells that "I want to use CS"
 → waits until other process "j" is free
 → just marks itself false telling that I am no longer using CS instead of forcefully handing over CS

M.E: ✓

Progress: X If both did their flag = true (wants CS) at some time; P1 will think P2 needs it & vice versa; so both wait forever.

B.W: ✓

□ Algo. 3 Peterson's (winner).

```
do {  
    flag[i] = true;  
    turn = i;  
    while (flag[i] && turn == j)  
        // CS  
    flag[i] = false;  
    // RS  
} while (1)
```

• Mixture of Algo 1 & 2
using both 'turn' and 'flag'.

• M.E: ✓

Progress: ✓ turn can only be i or j.

B.W: ✓

□ Bakery Algo: Peterson's is just for 2 processes; this is for >2 . Uses Arrival Number and Turn Number to allocate CS.

- Each process first gets arrival no. so if two process are getting their Turn/ticket no. at same time; decision is based upon (P0 or P1).
- Then they get actual 'turn no.' which gives them turn to use CS. ↓ Turn No means ↑ priority.

```
do {  
    choosing[i] = true;  
    numbering[i] = max(numbering[0], numbering[1], ..., numbering[n-1]) + 1;  
    choosing[i] = false;  
    for (j=0; j<n; j++) {  
        while (choosing[j])  
            while (numbering[j] != 0 && (numbering[j], j) < (numbering[i], i))  
                ;  
    }  
    // CS  
    numbering[i] = 0;  
    // RS  
} while (1)
```